Project Aegis

Designing Production-Grade Agent Systems with Closed-Loop Control Theory

Technical Guide | For engineering teams of 200–1,000

Author: Ares Sheng
Version 2.0 | April 2026
Classification: Enterprise Agent Architecture | Internal

Chapter 1 From Open-Loop Scripts to Closed-Loop Control Systems

When we kicked off Project Aegis, we ran an extensive evaluation and stress test against the mainstream open-source Agent frameworks—early ReAct-pattern implementations and their derivatives. The conclusion was unambiguous: every single one of them collapsed under real-world, production SRE (Site Reliability Engineering) workloads.

1.1 The Problem: Open-Loop Hallucination Cascades

The fundamental architectural flaw across the open-source ecosystem is that these frameworks are, at their core, open-loop script systems. The model emits a text instruction, the system parses and executes it, stuffs stdout back into the model's context, and then hopes the model understands what happened. In complex microservice fault diagnosis, this absence of genuine environmental state feedback causes hallucinations to compound at a terrifying rate.

> ⚠ **Hard Lesson: Open-Loop Hallucination Cascades**
>
> The moment the environment drifts even slightly—a command times out, a network hiccup occurs—an open-loop Agent will spiral into frantic retries or catastrophic long-tail operations. During our stress tests, one open-source framework issued 47 invalid kubectl delete commands within 15 minutes of a network blip.

1.2 The Architecture: Distributed Closed-Loop Control

To address this at the root, we decided to redefine the Agent in Project Aegis as a distributed closed-loop control system—architecturally analogous to a Kubernetes Controller. The large language model is nothing more than a non-deterministic compute node within the system.

The entire architecture is partitioned into three core planes:

◆ **Figure 1-1: Project Aegis Closed-Loop Architecture Overview**

```
graph TB
    subgraph Sensory["Sensory & State Plane"]
        SS["State Streaming\nK8s APIServer / Prometheus"]
        GDB["Graph DB (L2 Memory)\nMicroservice Topology Tree"]
        ES["Event Sourcing\nImmutable Append-Only Event Log"]
        SNAP["Snapshot Engine\nTimeline Memory Reconstruction"]
    end

    subgraph Control["Control Plane"]
        ACTOR["Actor State Machine Engine\nAsync Event-Driven"]
```

```
    ROUTE["Intent Router\nPerceive > Route > Plan > Yield > Callback"]
    BFT["BFT Consensus\nMulti-Model Quorum Voting"]
end

subgraph Actuation["Actuation Plane"]
    GATE["Zero-Trust Execution Gateway\nIntent Descriptor Parsing"]
    SB["Ephemeral Sandbox\nMinimal Capability Token"]
    EBPF["eBPF Probes\nSyscall-Level Interception"]
end

subgraph Feedback["Feedback & Resilience"]
    CK["Checkpointing\nDistributed K-V Persistence"]
    SAGA["SAGA Compensation\nReverse Rollback Chain"]
    DRIFT["Intent Drift Monitor\nObserve Intents, Not Tokens"]
end

SS --> GDB --> ACTOR
ES --> SNAP --> ACTOR
ACTOR --> ROUTE --> BFT --> GATE
GATE --> SB --> EBPF
EBPF -->|"Feedback Loop"| Feedback
Feedback -->|"State Realignment"| Sensory
```

↑ *Paste into mermaid.live or any Markdown renderer to view*

> **Source Audit Reference:** In parallel, we audited a state-of-the-art Agent system comprising nearly 5,000 source files. Its 42 built-in tools, 14-stage execution governance pipeline, and three-layer non-bypassable security mesh confirmed a harsh reality: production-grade Agents cannot be built with a few hundred lines of Python. However, its monolithic loop architecture still has limitations in distributed scenarios—precisely the frontier Aegis is designed to push past.

Chapter 2 The Sensory & State Plane: State Streaming and Temporal Memory

Conventional Agent development tends to fixate on “how to cram the context window full” or “how to summarize and compress text.” In real production environments, we learned the hard way that relying on an LLM’s context to maintain system state is an architectural disaster.

2.1 Core Design: The Context Storage Pyramid

We borrowed the storage hierarchy from computer architecture and applied it to Agent design. The LLM’s context window is treated as nothing more than an L1 Register—blazing fast, extremely precious, and equally prone to overflow.

Tier	Analogy	Implementation	Capacity	Access Latency
L1 Register	CPU Register	LLM Context Window	128K–200K tokens	<100ms
L2 Memory	RAM	Neo4j / JanusGraph (Graph DB)	Millions of nodes	5–50ms
L3 Disk	SSD/HDD	Event Log (Kafka) + Checkpoint (Etcd)	Unbounded append	10–200ms

> **Practical Advice: Technology Selection**
> For teams of 200–500 engineers, we recommend Neo4j Community Edition as the L2 store—its Cypher query language has a gentle learning curve. Teams of 500–1,000 should consider JanusGraph with a Cassandra backend for better horizontal scalability. For L3, use Kafka or Pulsar for the Event Log, and Etcd or Redis Streams for Checkpointing.

2.2 State Streaming

When Aegis connects to a live cluster, it does not read state through text. Instead, it subscribes to state streams from the K8s APIServer and Prometheus, maintaining a real-time microservice topology tree inside the graph database. The LLM only pages in a local subgraph from the graph database into its context register when it needs to reason over it.

TypeScript: State Streaming Implementation

```typescript
// State Streaming: Watch K8s pod state changes, update graph DB in real time
async function watchPodState(kubeClient: KubeClient, graphDb: GraphDB) {
  const watcher = kubeClient.watch('/api/v1/pods');

  watcher.on('MODIFIED', (pod: Pod) => {
    // Write pod state change to graph database
    graphDb.upsertNode({
      id: pod.metadata.uid,
      type: 'Pod',
```

```
    props: {
      name: pod.metadata.name,
      status: pod.status.phase,
      restartCount: pod.status.containerStatuses?.[0]?.restartCount,
      lastTransition: Date.now(),
    },
  });

  // Simultaneously append to Event Log (L3 tier)
  eventLog.append({
    type: 'POD_STATE_CHANGE',
    timestamp: Date.now(),
    payload: { podId: pod.metadata.uid, newStatus: pod.status.phase },
  });
 });
}
```

2.3 Hard Lesson: Event Sourcing Saved Us

> ⚠ **Hard Lesson: Production Incident Postmortem**
>
> Our initial naive implementation caused a severe production incident. While investigating a complex cross-availability-zone network failure, Aegis ran through 30+ rounds of deep retrieval, and its L1 register was forced to evict aggressively. The Agent forgot the original P0 OOM alert that triggered it and began "sleepwalking" through irrelevant log files. This single incident stretched the expected 15-minute recovery to over 2 hours.

To address this at the root, we introduced Event Sourcing. All critical state mutations are serialized into an immutable, append-only event stream. When the L1 register suffers a cache miss or drifts significantly from baseline, the state plane replays the Event Log to instantly reconstruct a time-series memory snapshot, forcefully realigning the Agent's cognitive baseline.

> **Source Audit Reference:** Our audit found that the SOTA system employs a 4-level eviction compression strategy (Snip → Micro → Collapse → Auto Compact) to manage context overflow, and uses a SYSTEM_PROMPT_DYNAMIC_BOUNDARY marker to physically separate the static and dynamic portions of the system prompt, maximizing API cache hit rates (saving approximately 73% on prompt costs). This is excellent engineering—but it is still compression within the L1 tier. Aegis's pyramid model redefines the state storage hierarchy from the ground up.

Chapter 3 The Control Plane: Actor Model and Multi-Path Consensus

3.1 Core Design: An Event-Driven Actor Engine

In our early prototypes, we tried driving Agent reasoning with recursive code and busy loops. Both proved extremely fragile in distributed systems.

> **Source Audit Reference:** The SOTA system's core engine is a 1,729-line async generator built on a while(true) + state object pattern, containing 9 distinct state transition trigger points (including circuit-breaker retries, model degradation fallback, token budget nudging, and stop-hook interception). An earlier version used recursive self-calls and was refactored after stack overflows in long sessions. This proved that state machines are fundamentally more resilient than recursion—we refer to it as "the engineering embryo of a state-machine kernel."

Project Aegis builds on this foundation by introducing an event-driven engine based on the Actor Model. Each logical Agent is an independent Actor with its own mailbox:

1. **Perception:** Receive monitoring alerts as Events and enqueue them in the Actor's mailbox.
2. **Routing:** Transform raw Events into standardized, strongly-typed intent structures.
3. **Planning:** Generate a diagnostic decision tree for the current investigation.
4. **Yield/Await:** Dispatch long-running actions, then immediately suspend the Actor to free compute resources.
5. **Callback:** When the actuation plane completes an action, it wakes the Actor through the event bus to continue.

This fully asynchronous, non-blocking lifecycle allows a single node to orchestrate hundreds or thousands of concurrent investigation tasks.

> **Practical Advice: Technology Selection**
> For teams of 200–500, we recommend a Node.js-based Actor implementation (e.g., xstate + BullMQ). Teams of 500–1,000 should consider Elixir/OTP or Akka (JVM), which provide native Actor supervision trees and fault migration capabilities.

3.2 Multi-Path Consensus: Multi-Model Decisions Under a BFT Metaphor

When it comes to high-risk operations—restarting a core transaction database, blocking a specific IP—we trust no single model's judgment. We borrowed from the Byzantine Fault Tolerance (BFT) paradigm: high-risk intents are multicast to three independent, specialized small language models. Only when a 2/3 quorum votes to approve does the control plane allow the operation to proceed.

TypeScript: BFT Consensus Gate

```
// BFT Consensus Gate
```

```
async function consensusGate(intent: IntentDescriptor): Promise<boolean> {
  // Multicast the intent to 3 independent, specialized risk models
  const votes = await Promise.all([
    riskModel_DB.evaluate(intent),       // Database risk specialist
    riskModel_Network.evaluate(intent),  // Network risk specialist
    riskModel_Blast.evaluate(intent),    // Blast radius estimator
  ]);

  const approvals = votes.filter(v => v.decision === 'APPROVE').length;
  const quorum = Math.ceil(votes.length * 2 / 3); // 2/3 supermajority

  if (approvals >= quorum) {
    return true; // Proceed
  }

  // Quorum not met: escalate to human approval
  await escalateToHuman(intent, votes);
  return false;
}
```

Chapter 4 The Actuation Plane: Zero-Trust Execution Gateway and eBPF Sandboxing

There is a rule inside our team that we enforce with near-religious severity: “Every string emitted by a large language model is toxic until proven otherwise.”

4.1 Core Design: Intent Descriptors and the Execution Gateway

The LLM in Aegis has absolutely no execution privileges—it cannot touch an API key or a kubeconfig. Its output is parsed exclusively into a strongly-typed Intent Descriptor.

TypeScript: Execution Gateway Interface

```typescript
// Core principle: Non-deterministic models emit Intents only.
// Deterministic gateways own actual Actions.
interface IntentDescriptor {
  readonly actionType: 'READ_METRICS' | 'MUTATE_STATE' | 'ESCALATE_TO_HUMAN';
  readonly targetResource: string;   // e.g., 'pod/payment-svc-xyz'
  readonly payload: unknown;
  readonly riskLevel: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL';
  readonly consensusProof?: ConsensusSignature; // Required for high-risk ops
}

interface IExecutionGateway {
  // Gateway entry point: transform non-deterministic intent
  // into a controlled, sandboxed physical execution
  dispatch(intent: IntentDescriptor): Promise<ExecutionResult>;

  // Physical isolation: spin up an ephemeral sandbox per intent
  provisionEphemeralSandbox(intent: IntentDescriptor): SandboxContext;

  // Kernel-level enforcement: mount eBPF syscall interceptors
  injectKernelProbes(sandboxId: string, rules: SyscallRule[]): void;
}
```

4.2 How the eBPF Sandbox Works

Upon receiving an Intent, the execution gateway spins up a short-lived ephemeral sandbox container and injects a minimal-privilege Capability Token. At the node level, eBPF probes are deployed. If the LLM hallucinates and the sandbox attempts to execute rm -rf / or initiate a network connection to a non-whitelisted IP, the eBPF interceptor kills the syscall at the Linux kernel level and emits an anomaly event.

> **Source Audit Reference:** Our audit observed that the SOTA system's tool execution is governed by a 14-stage pipeline (from Zod schema validation through permission decisions to telemetry recording), employs fail-closed defaults (if a tool forgets to declare concurrency safety, the system serializes it), and uses a three-layer non-bypassable security mesh (a Hook's allow cannot override a Settings deny). This represents the pinnacle of application-layer governance. Aegis pushes governance down to the kernel layer via eBPF, achieving physical isolation rather than logical isolation.

4.3 Streaming Execution and Intent Pre-Parsing

In distributed operations scenarios, even seconds of latency can allow a fault to propagate. We implemented a critical architectural optimization: Intent Pre-parsing. The gateway begins cold-starting the sandbox container and pre-fetching network resources in parallel as soon as it detects the first valid instruction fragment in the model's streaming output—rather than waiting for the full response to complete. This "instruction–environment parallel loading" design reduced Aegis's end-to-end response latency by 40%.

> **Practical Advice: Implementation Path**
> Smaller teams can start with Docker + seccomp profiles as a first-generation sandbox—there is no need to deploy eBPF on day one. Once system load exceeds 100 QPS, upgrade to eBPF. Intent pre-parsing can be implemented with a streaming JSON schema parser such as the jsonparse2 library.

Chapter 5 The Feedback & Resilience Plane: Self-Healing After Failure

The true test of a system's design is not how it behaves when everything works. It is how it recovers when things break. If a network hiccup or API circuit-break occurs mid-reasoning, the naïve approach of "start over from scratch" leads to duplicated operations and state corruption.

5.1 Checkpointing: Persistent State Across Failures

We introduced checkpointing into the control plane. Every decision round, every generated Intent, and every piece of actuation-plane feedback is synchronously persisted to a distributed key-value store. If an Actor node fails and migrates, the new node reads the latest checkpoint and instantly restores its full context and execution stack—achieving lossless retry.

5.2 The SAGA Pattern: Compensating Transactions

LLM-driven instructions often span multiple heterogeneous systems. If Aegis executes a sequence like "scale up replicas → update RBAC → reconfigure ingress" and the final step fails, the system cannot simply leave behind a half-finished mess.

TypeScript: SAGA Compensating Transaction

```typescript
// SAGA Compensating Transaction Implementation
interface SagaStep {
  name: string;
  execute: () => Promise<StepResult>;
  compensate: () => Promise<void>;  // Every Action MUST pair with a Compensation
}

async function executeSaga(steps: SagaStep[]): Promise<SagaResult> {
  const completed: SagaStep[] = [];

  for (const step of steps) {
    try {
      await step.execute();
      completed.push(step);
    } catch (error) {
      // Chain broken: roll back all completed steps in reverse order
      console.error(`Step "${step.name}" failed, rolling back...`);
      for (const done of completed.reverse()) {
        await done.compensate();
```

```
      }
      return { success: false, failedAt: step.name, error };
    }
  }
  return { success: true };
}

// Usage example
const migrationSaga: SagaStep[] = [
  {
    name: 'scale-up-replicas',
    execute: () => kubectl.scale('deployment/api', { replicas: 5 }),
    compensate: () => kubectl.scale('deployment/api', { replicas: 3 }),
  },
  {
    name: 'update-rbac',
    execute: () => kubectl.apply(newRbacPolicy),
    compensate: () => kubectl.apply(originalRbacPolicy),
  },
  {
    name: 'update-ingress',
    execute: () => kubectl.apply(newIngressConfig),
    compensate: () => kubectl.apply(originalIngressConfig),
  },
];
```

Source Audit Reference: The SOTA system's runAgent.ts (973 lines) executes a complete cleanup chain at the end of every sub-Agent lifecycle: transcript recording, metadata writes, Perfetto trace closure, shell process termination, hook cleanup, file state cleanup, and MCP connection teardown. This is a textbook example of Day-2 operational awareness. Aegis upgrades the cleanup chain into full SAGA compensation—not just cleanup, but rollback.

Chapter 6 Lessons from Production: Five Principles for Production-Grade Agent Architecture

From its earliest Demo to supporting core production workloads, and informed by our deep source-level audit of the industry's most advanced Agent system, Project Aegis has distilled five principles that belong in every engineering team's architectural playbook.

Principle 1: State Decoupling

Never store system state inside the LLM's context window. The model is a compute unit, not a storage unit. State must be externalized to an auditable, rollback-capable system—a graph database, an event stream, or both.

Principle 2: Physical Separation of Intent and Actuation

Never grant an LLM the ability to call APIs directly. There must be a zero-trust gateway in the architecture. The model's sole job is to emit intents. Physical operations must be executed by a controlled, sandboxed proxy.

Principle 3: Surround Non-Determinism with Determinism

The model's output is non-deterministic by nature. But the defense chain around it—input validation, permission checks, transaction rollback—must be hardcoded and deterministic. This "deterministic shell" is the only ticket to industrial-grade Agent systems.

Principle 4: Observe Intents, Not Tokens

When building telemetry, counting how many tokens the model emits is meaningless. What matters is intent drift rate and action success rate. You need to measure the gap between what the Agent intended to do and what it actually accomplished.

Principle 5: The Lifecycle Cleanup Chain Must Be Complete

Getting a Demo to work is Day 1. Every Agent and sub-Agent must execute a complete cleanup chain on termination: state persistence, process cleanup, connection release, resource reclamation. An Agent system without a cleanup chain will leave behind zombie processes and corrupted state by Day 2.

Conclusion

Project Aegis demonstrates that Agent systems are far more than a Prompt Engineering exercise.

They are a rigorous triumph of distributed systems design and closed-loop control theory. We do not need a smarter model. We need a more robust system framework to constrain the model.

This document is part of the Project Aegis open-source architecture series.